

Klasse:	Arbeitsblatt 2 – Wichtige Hamsterbefehle	
Fach:		Datum:

1. Testbefehle

Um Fehler bei der Ausführung von `vor()`; `nimm()`; und `gib()`; zu vermeiden, gibt es die sogenannten Testbefehle.

- Befehl: `vor()`;
Problem: Ein Fehler tritt auf, wenn der Hamster vorwärts gehen möchte, aber es steht eine Mauer im Weg oder er befindet sich am Rand.
Lösung: `vornFrei()`; : Dieser Testbefehl prüft, ob der Weg vor dem Hamster frei ist, also ob er vorwärts gehen kann, ohne auf eine Mauer oder einen Rand zu stoßen.
- Befehl: `nimm()`;
Problem: Ein Fehler tritt auf, wenn der Hamster versucht, ein Korn aufzunehmen, aber keins vorhanden ist.
Lösung: `kornDa()`; : Dieser Testbefehl prüft, ob auf dem Feld, auf dem sich der Hamster gerade befindet, ein Korn liegt oder nicht.
- Befehl: `gib()`;
Problem: Ähnlich wie bei `nimm()`; besteht ein Problem darin, dass der Hamster versucht, ein Korn abzulegen, obwohl er gar nichts im Maul hat. Dies führt zu einem Fehler.
Lösung: `maulLeer()`; : Dieser Testbefehl prüft, ob der Hamster ein Korn im Maul hat.

2. Operator Verneinung (Negation)

Alle Testbefehle selbst sind Unterprogramme, die nach dem Aufruf einen sogenannten Rückgabewert zurückliefern. Diese Rückgabewerte sind entweder 'wahr' (`true`) oder 'falsch' (`false`).

Manchmal benötigt man für die Programmierung allerdings genau das Gegenteil der Testbefehle. Also anstatt `vornFrei()`; benötigt man `nicht_vornFrei()`; Dieser Befehl existiert allerdings nicht. Er kann aber durch den Operator `!` erzeugt werden.

Wenn `vornFrei()`; den Wert `true` liefert, dann liefert `!vornFrei()`; den Rückgabewert `false` und umgekehrt.

Klasse:	Arbeitsblatt 2 – Wichtige Hamsterbefehle	
Fach:		Datum:

3. if/else-Anweisung

Die Testbefehle werden **immer** zusammen mit der sogenannten 'if-Anweisung' (wenn-Anweisung) ausgeführt.

Die if-Anweisung ermöglicht es, Entscheidungen aufgrund der Rückgabewerte der Testbefehle zu treffen:

```
if (vornFrei()) {
    // Führe Aktion aus, wenn der Weg vorwärts frei ist
}

if (maulLeer()) {
    // Führe Aktion aus, wenn das Maul des Hamsters leer ist
}

if (kornDa()) {
    // Führe Aktion aus, wenn auf dem Feld ein Korn liegt
}
```

Beispiele, wie man alles zusammen verwendet

a)

```
if (vornFrei())
{
    vor();
    linksUm();
}
```

b)

```
if (!maulLeer())
{
    gib();
}
```

c)

```
if (!kornDa())
{
    vor();
}
else
{
    nimm();
}
```

Die if/else-Anweisung ist wie ein Wegweiser, der dem Hamster sagt, was er tun soll, abhängig von den Informationen, die er hat. Wenn die Bedingung des Testbefehls wahr ist, geht er in die if-Richtung, sonst in die else-Richtung.

```
if (maulLeer()) {
    // Führe Aktion aus, wenn das Maul des Hamsters leer ist
}
else
{
    // Führe Aktion aus, wenn das Maul des Hamsters nicht leer ist
    // d.h. wenn er mindestens ein Korn im Maul hat
}
```

Wichtig: if-Anweisungen sind KEINE Schleifen. Der Begriff if-Schleife ist deshalb unsinnig!!!
--

Klasse:	Arbeitsblatt 2 – Wichtige Hamsterbefehle	
Fach:		Datum:

4. while-Schleife

Die `while`-Schleife ist eine Wiederholungsanweisung, die den Hamster dazu bringt, eine Aufgabe immer wieder auszuführen, solange die abgeprüfte Bedingung (Testbefehle) wahr ist.

Sobald die Bedingung nicht mehr wahr ist, hört er auf, diese Aufgabe zu wiederholen, und fährt mit den Anweisungen **nach** der Schleife fort.

Eine Schleife hilft dem Hamster, wiederholte Aktionen zu automatisieren und eine Aufgabe so oft auszuführen, wie es notwendig ist.

Beispiel

```
void main() {
    while (!kornDa()) {
        vor();
    }
    nimm();
}
```

In diesem Beispiel bewegt sich der Hamster vorwärts, solange kein Korn auf dem aktuellen Feld liegt. Sobald er auf einem Feld steht, auf dem ein Korn liegt, wird die Schleife beendet. Anschließend nimmt er das Korn auf.

5. Prozeduren

Manchmal stellt man fest, dass Befehle zu fehlen scheinen. So existiert zwar ein `linksUm()`; jedoch fehlt `rechtsUm()`. Die Prozedur `rechtsUm()` existiert nicht, weil sie nicht notwendig ist. Wir können die Drehung um 90 Grad nach rechts erreichen, indem wir die Prozedur `linksUm()` dreimal aufrufen.

Solche neuen Befehle kann der Programmierer mit alten Befehlen also selbst zusammenbauen.

```
void rechtsUm() {
    linksUm();
    linksUm();
    linksUm();
}
```

Jetzt steht im Programm auch der Befehl `rechtsUm()` zur Verfügung. Auf gleiche Weise könnte man jetzt z.B. den Befehl `kehrt()` entwickeln.

Klasse:	Arbeitsblatt 2 – Wichtige Hamsterbefehle	
Fach:		Datum:

Zusammenfassung

Zusammenfassung der bisherigen Hamstersprache:

Befehl	Funktion
<code>vor();</code>	Bewegt den Hamster ein Feld nach vorne
<code>linksUm();</code>	Dreht den Hamster um 90 Grad nach links
<code>nimm();</code>	Nimmt ein Korn auf, falls eins vorhanden ist
<code>gib();</code>	Legt ein Korn ab
<code>kornDa()</code>	Überprüft, ob auf dem aktuellen Feld ein Korn liegt. Liefert <code>true</code> oder <code>false</code> zurück.
<code>vornFrei()</code>	Überprüft, ob das Feld vor dem Hamster frei ist. Liefert <code>true</code> oder <code>false</code> zurück.
<code>maulLeer()</code>	Überprüft, ob der Hamster ein Korn im Maul hat. Liefert <code>true</code> oder <code>false</code> zurück.
<code>!</code>	Verneinung, kann auf alle Testbefehle angewendet werden. Liefert das Gegenteil des Ergebnisses des Testbefehls
<pre>while (Testbefehl) { Anweisung1 Anweisung2 . . }</pre>	Solange der Testbefehl wahr liefert, werden die Anweisungen ausgeführt. Wenn der Testbefehl falsch liefert wird die Schleife verlassen.
<pre>void rechtsUm() { linksUm(); linksUm(); linksUm(); }</pre>	Neues Unterprogramm (Prozedur) mit dem Namen <code>rechtsUm()</code> . Das Unterprogramm wird im Hauptprogramm mit <code>rechtsUm();</code> aufgerufen.
<code>// Text wird nicht beachtet</code>	Kommentarzeile

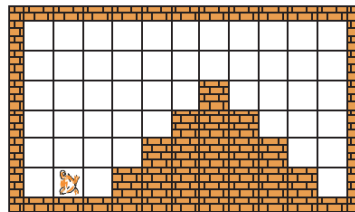
Klasse:	Arbeitsblatt 2 – Wichtige Hamsterbefehle	
Fach:		Datum:

Übungen

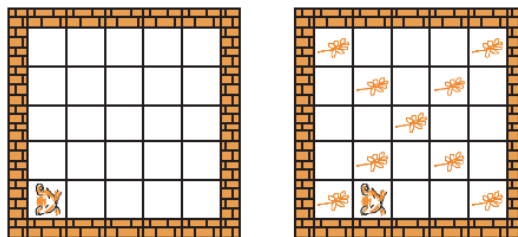
- Häufig ist es nützlich kleine Prozeduren vor dem eigentlichen Programm zu schreiben. So wie `rechtsUm()` ; oder `kehrt()` ; . Entwickle folgende Prozeduren und teste sie:

- `erklimmeStufe();`
- `nimmAlle();`
- `gibAlle();`

- Versuche das folgende Problem mit Prozeduren zu lösen! Der Hamster steht genauso wie im Bild zu sehen. Er soll den Berg erklimmen.



- Versuche das folgende Problem mit Prozeduren zu lösen! Der Hamster soll in allen Feldern der beiden Diagonalen jeweils genau ein Korn ablegen. Allerdings sieht diesmal das Hamster-Territorium so aus, wie in der folgenden Abbildung. Der Hamster habe anfangs genau 9 Körner im Maul.



- Versuche das folgende Problem mit Prozeduren zu lösen! Der Hamster hat Schiffbruch erlitten und ist auf einer einsamen Insel gestrandet. Er hat zum Glück noch 100 Körner dabei. Um Flugzeuge auf sich aufmerksam zu machen, will er aus den Körnern die Buchstaben SOS ablegen (siehe Abbildung). Helfen Sie ihm dabei. Der Hamster steht anfangs in der linken unteren Ecke des Territoriums mit Blickrichtung Ost.

